

Synthesizing Staged Updates between Verified Controllers

ANONYMOUS AUTHOR(S)

Verified controllers for old and new software versions leave an open problem: how to complete the transition while component states and active requirements change. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) takes local transfers, requirement lifetimes and initial obligations as a contract and synthesizes a staged update policy. In the model, a winning policy safely completes after finitely many events from every reachable old state under every permitted outcome, then hands control to the supplied new controller without changing its continuation. Commands and handover require states with no enabled uncontrollable event. We prove contract/game correspondence and specialize established lazy reachability to return a policy and progress rank or a losing-region certificate. Merging commands enables or prevents completion in constructed examples, but changes no decision in 54 comparisons derived from existing models. On the same 27 adapted contract games, our implementation decides 26 within 1,200 s with lazy exploration, versus 17 for eager successors, 24 for update-first exploration and 14 for full construction. Update-first is sometimes faster. Continuous service and elapsed-time bounds are not guaranteed.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Synthesizing Staged Updates between Verified Controllers. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

A verified old controller and a verified replacement do not tell us how to move between them. During an update, some components may already use the new version while others retain the old one. Requirements also change: an old rule may need to remain active until a particular step, and a new rule may begin with an obligation inherited from the current state [3, 7, 15, 38].

FG-DUCS generates a completing update policy from local choices. The modeler supplies permitted component transfers, the lifetimes of requirements, their initial obligations and the old and new controllers. Synthesis chooses a policy that works from every reachable old state and responds to every environment outcome. At handover, it preserves the allowed continuation of the selected new-controller state; satisfying new safety goals alone need not preserve that continuation (Section 3).

The intended setting permits maintenance pauses. Update control replaces the old controller's scheduling, while active requirements continue to constrain behavior. Commands and handover wait until no uncontrollable (UC) event is enabled. Requirements that must hold across the whole transition are declared as interval requirements. This gives a precise boundary: an environment that can avoid such quiescence forever prevents completion. The guarantee counts events, not elapsed time or continuous service.

Dynamic Update Controller Synthesis (DUCS) [33, 34] and GR(1) updating [2] already synthesize update or bridge control. FG-DUCS fixes both endpoint controllers and generates the intervening game from local transfers and individual requirement boundaries. Section 2.3 compares these input interfaces on the same example. The contribution is the construction and its guarantees; the exploration principles are established, and modeling effort reduction has not been measured.

We develop and assess this construction as follows:

C1: A local contract and its generated game. We define the generated game, prove local-history correspondence and selected-endpoint preservation, and give an invariant-initialization criterion (Sections 3 and 4.1).

C2: Boundaries of update granularity. FG-DUCS examples and merging obstructions explain when separate choices are necessary and when they prevent completion (Sections 2 and 5).

C3: A policy with checkable evidence. Sound and complete lazy synthesis returns a completing policy and rank or a losing-region certificate (Section 6).

C4: Implementation evidence. Finite reference games check our implementation; exploration is evaluated on 27 adapted contracts; constructed comparisons test granularity (Section 7).

Our public replication package [4], prepared for this paper, includes the cited Technical Appendix A–P and Supplement S1–S5 as Parts I and II of `supplementary_material.pdf`. Data Availability gives versioned links.

2 Why Local Transfers and Requirement Lifetimes Matter

Existing approaches can represent staged updates, but the coordination must appear in their supplied update model. The following examples isolate two decisions that FG-DUCS generates from local inputs: when to move and replace a component, and when to start or stop a requirement. Section 2.3 then compares the supplied and generated structures with existing interfaces.

2.1 Cell: Move, Replace, and Discharge an Obligation

Cell is a constructed two-station model with one workpiece, motivated by production-cell models [23, 34]. Either station can be replaced only when empty. The new version of station *B* requires inspection before a workpiece leaves; the old rule forbids inspection. The input requires the new rule to start before the old rule stops, so the policy must manage a temporary overlap.

Table 1 separates the choices supplied by the modeler from the coordination left to synthesis. A policy may move the workpiece, replace an empty station, switch requirement boundaries and inspect when the active rules permit it. Inspection may repeat forever in the plant, so safety alone is insufficient.

Table 1. Cell’s local inputs and the coordination they require. The complete finite input is in Technical Appendix B.

Supplied constraint	Consequence for an update policy
Replace only an empty station	Move the workpiece between the two replacements.
Always retain one workpiece	A simultaneous replacement of both empty stations is impossible.
Start the new rule before stopping the old	Overlap the rules, then stop the inspection ban before inspecting.
New rule: inspect before departure from <i>B</i>	Starting while <i>B</i> holds the workpiece installs a pending inspection duty.
Handover only with inspection clear	Clear any pending duty before entering the supplied new controller.

Both possible old workpiece positions are update entries. The initializer installs inspection-pending exactly when *B* holds the workpiece, and inspection-clear otherwise. The loadable new-controller states require inspection-clear. Technical Appendix B gives all component and monitor transitions, endpoint tuples and initializer entries.

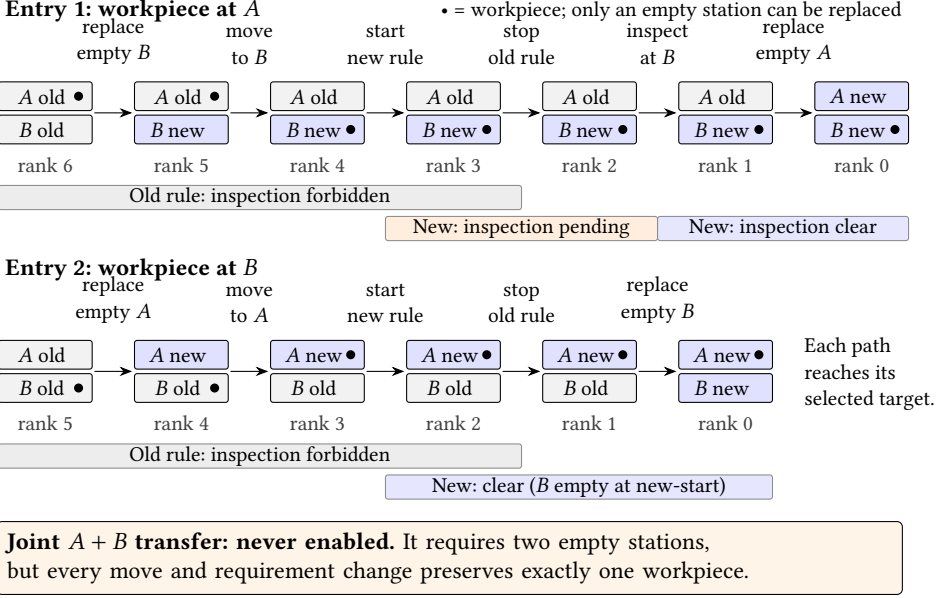


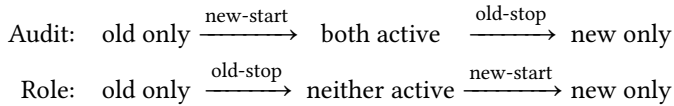
Fig. 1. A complete local policy from both old entries. Bands mark active rules and new-rule inspection memory; new-start precedes old-stop. Each arrow has one action and one outcome, decreasing rank toward its selected new target. Joint transfer merges these local commands with endpoints and requirements unchanged; this is not a DUCS encoding.

The generated policy. Figure 1 gives the whole policy. Starting with A occupied, it replaces empty B, moves the workpiece to B, starts the new rule, stops the old rule, inspects, and replaces empty A. Starting with B occupied, it first replaces A and moves the workpiece there; the new rule then starts without an inspection debt. Every displayed event has one outcome and other controls are disabled.

The two paths contain 13 states and 11 edges. Their remaining lengths decrease from six or five to zero, proving completion as well as safety. A joint replacement cannot begin because it requires both stations empty. The example therefore shows why synthesis must coordinate local choices instead of merely check each endpoint. Technical Appendix B and S1 give the complete closure argument.

2.2 Audit and Role Lifetimes

Policy(2) denotes this constructed example's two old/new requirement pairs: audit and access roles. Four bits record active rules a_o, a_n, r_o, r_n . The interval invariant $(a_o \vee a_n) \wedge \neg(r_o \wedge r_n)$ requires opposing boundary orders:



The role gap follows from the declared interval invariant; access monitors alone do not require it. The new audit monitor starts unlogged and requires a log before service. Each endpoint alternates its own log and service events and permits only its version's access role. With an identity component transfer, the saved policy first performs service if the old audit is logged, starts new audit,

stops old audit and role, transfers, and starts the new role. Service is optional; this policy reaches a compatible new endpoint from both old states, with rank at most six.

If the starts are merged and the stops are merged, the first start overlaps roles and the first stop removes audit coverage. Ordinary events and transfer preserve the four activity bits, so neither old entry has a completing merged policy. Technical Appendix C gives the complete finite input, policy and losing closure. This compares merged and separate FG-DUCS requirement boundaries.

2.3 Existing Update Interfaces

Table 2. Supplied inputs and generated results: DUCS [34, Defs. 4.1–4.3, Sec. 5], GR(1) updating [2, Def. 3.1, Sec. 4], and FG-DUCS.

Interface	Supplied inputs	Generated structure and successful result	Progress and continuation
DUCS	Old control; old/new environments E, E' and goals; global transfer relation R ; transition requirement T	Update environment and goal; controller C' and map H covering every old-controller state	Eventual stop/start/reconfigure after hotSwap; C' enforces new goals.
GR(1) updating	Old/new GR(1) specifications and switching condition	New controller C_2 , bridge and switching bookkeeping	This C_2 runs unchanged after the bridge.
FG-DUCS	Fixed components/controllers; component transfers; monitors, initializers and precedence; loadable targets	Mixed-version and boundary game; policy, rank and handover map for every old entry and permitted outcome	Supplied new closed loop is preserved from the selected compatible target.

Earlier endpoint reconnection. On-the-fly DUCS (O-DUCS) separates update and post-update analysis [18, Sec. 3]. The archived fork in S4.8 synthesizes a new controller, matches component and new-safety states, and copies its transitions [4]. Identity with the 2024 release is unestablished.

Independent requirement boundaries. DUCS’s T constrains one global stop/start pair, including overlap, gaps and their duties [34, Def. 4.1]. FG separates individual stops and starts/initializers.

Cell through native interfaces. DUCS permits staged reconfiguration inside E' [34, Sec. 4.2]. Technical Appendices H–I give chosen Cell encodings: DUCS adds eventual handover beyond its safety- T implementation [34, Sec. 5.3]; GR(1) may restrict endpoint edges when synthesizing C_2 . These controllable, deterministic witnesses establish neither general translation nor cost advantage (S5).

3 Defining an Update Contract

3.1 Supplied Choices

Finite LTSs and safety monitors. A labeled transition system (LTS) $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{uc} \uplus \Sigma_c$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [25]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing errors for violating prefixes [1, 22]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \parallel_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* t_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed. Section 4.1 defines history-based and activation-scoped interpretations. For $r \in R^{\text{upd}}$, $\text{dom } t_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events leaves observation alphabet $\bigcup_i S_{e_i^v}$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} t_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These controllable events transfer components, stop old monitors or start new ones, respectively, and are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome.

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. An eligible contract satisfies the finiteness, typing, totality and non-error conditions above; dependencies form a strict partial order, and endpoint controller alphabets cover environment labels. Eligibility is structural; Section 4.1 justifies initializer meanings.

3.2 States, Transitions, and Completion

States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i^v}$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ copies z 's component and old-monitor states, installs interval initializers, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The update policy replaces C^{old} and its scheduling restrictions. Declared old monitors remain until their stops; undeclared objectives, including old liveness, are not preserved. Duties needed throughout belong in interval monitors. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

All-entry coverage forbids waiting for a favorable old state.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

Thus even a local replacement waits until the active-component product has no enabled uncontrollable event. A finite tool operation may drain to such a state; a perpetual polling loop may prevent it. This global *uncontrollable quiescence* differs from transaction quiescence [21] and tranquility [42]. Ordinary controls may coexist with UC. The following rules define Post ; unmentioned coordinates stay unchanged. “Advance” means $\mu'(t_r) = \eta_r(\mu(t_r), a)$ for the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } t_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = t_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals*. An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$\mathcal{Q}_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

At a goal, interval obligations end and matched new monitors continue. Handover disables ordinary controllables and installs $\kappa(q)$'s controller state, outside Σ and the rank. UC cannot compete; compatible targets may have different continuations.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : \mathcal{Q} \setminus \mathcal{Q}_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_{\sigma}(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_{\sigma}(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches $\mathcal{Q}_{\text{goal}}$ after finitely many events. This *finite completion* counts events and assumes no fairness. Winning policies exclude controllable self-loops during update; a reached UC self-loop makes a state losing.

Ordinary actions can progress, as Cell’s moves and inspection do. Handover restores the supplied controller’s full matched behavior.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_\sigma(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_\sigma(q)$.

The FG-DUCS problem. Decide whether one policy safely completes the eligible game from every entry under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Why the Generated State Represents the Contract

A *local update history* $\ell = z(a_1, o_1) \cdots (a_n, o_n)$ records the old entry, events and realized local successors. Components follow their active-version transitions and transfers; monitors fold their declared active intervals. Commands obey precedence, guards and UC quiescence. A source policy selects controllable labels, retains every UC and permitted outcome, and safely completes every maximal history at a matching quiescent target. Technical Appendix A defines these contract operations on histories.

PROPOSITION 3.1 (CONTRACT/GAME CORRESPONDENCE). *For an eligible contract, map each local update history to its current component tuple, active monitor memories and unexecuted commands. Histories with the same image admit the same next events and all the same local outcomes, agree on safety, and have the same handover targets. This map gives a label- and outcome-preserving correspondence between local update histories and paths of the generated game. A history-dependent policy satisfying the source obligations exists exactly when a winning policy exists in that game.*

PROOF. Fold a source history into its current tuple $F(h)$. At entry this is $\text{Init}(z)$. Each ordinary step uses the same participating component edges and full Cartesian product of outcomes; each transfer uses the same quiescence/order tests and g_i , advances active monitors and consumes its command. Stop removes its own monitor before observation; start installs its initializer after observation by existing monitors. Thus every label and outcome has exactly the same successor image, and safety and matching targets agree.

Induction maps source histories to game paths. Conversely, choose an old tuple witnessing the entry and lift each edge by this one-step correspondence, also for infinite paths. A game policy lifts through F ; applying a source policy to consistently lifted game histories gives a history-dependent game policy. Both directions preserve UC events, all selected-label outcomes, unsafe prefixes, deadlocks and infinite avoiding runs. Finite reachability attractor layers yield a state-based winner exactly when any history-dependent winner exists (the independent memoryless-policy lemma in Technical Appendix A), completing the equivalence. $\square$

This proves correspondence for the supplied contract; requirement meaning and Java frontend correctness remain separate obligations.

PROPOSITION 3.2 (PRESERVATION OF THE SELECTED ENDPOINT). *For a returned WIN certificate and reached goal q , the new-phase submodel reachable after handover at q is label-preserving isomorphic to CL^{new} rooted at $\kappa(q)$. Hence their finite and infinite ordinary-event traces coincide.*

PROOF. Handover retains matching components and monitors and installs the selected controller state. Each new-phase state copies an endpoint state with exactly its labeled transitions, without further policy restriction or return to updating. This bijection preserves and reflects edges, hence finite and infinite paths. $\square$

Continuation properties established at the selected state are preserved; target selection can change which continuation applies.

Why matching and reconnection both matter. Cell's prefix $\rho_B, m_B, \text{start}, \text{stop}, \rho_A$ finishes every command with inspection pending. Physical matching alone admits clear target z_B , whose m_A violates that duty; monitor matching excludes the reset. Conversely, disabling controllable loop b in a one-state endpoint with loops a, b and goal *true* preserves legality, deadlock freedom and safety [34, Defs. 3.9–3.10, 4.3], but loses b -continuations. FG's fixed-endpoint contract requires preserving those continuations.

Technical Appendix G gives the endpoint interface and conditions for reusing a certificate when endpoints change.

4 What a Winning Policy Guarantees

The game proves safe completion for the supplied monitors. To understand what this means for a requirement, we must first identify the obligation installed when that monitor starts. We then state what is needed to carry the model guarantee into an execution. Table 3 provides this map.

Table 3. Guarantee scopes and their premises. Table 5 maps RQ2 checks to these scopes.

Scope	What follows	Additional premise or boundary
Game	Safe finite completion for all entries/outcomes; matched handover	The supplied transition and endpoint rules (Theorem 6.1)
Invariant from start	Truth at activation and while active	Correct predicates/observations and initializer conformance; ACT below
Safe prior histories	Continuations preserve obligations of the admitted safe histories	Residual inclusion; an empty history scope makes inclusion vacuous
From update entry	Interval monitors reflect obligations from admission onward	Correct boundary state and entry interpretation; S3 gives sufficient conditions
Execution	Handover within the entry's event rank; selected endpoint continues	A1–A4; no elapsed-time bound or guarantee of platform conformance

4.1 Giving Requirements Their Intended Lifetimes

An initializer assigns the obligation present at activation; selecting any non-error state is insufficient.

An exact activation obligation. Cell's owed-inspection bit starts at 1 exactly when B holds the workpiece. Event m_B sets it, j clears it, and m_A is forbidden while it is 1; other labels preserve it. Encoding 0 by c and 1 by d makes every monitor transition identical to this rule, including rejection of forbidden departure. Initialization and suffix induction prove exact enforcement. Figure 1's first path starts with debt and clears it by ordinary inspection; the second starts clear.

Invariants that begin at activation. Activation-scoped interpretation ACT checks a pure invariant $\Box\varphi$ at the actual post-start observer vector and after every later label. It retains factual fluent values, not a past-violation bit. For deterministic observers with total update $d : V \times \Sigma \rightarrow V$, define

$$\text{ACT}_\varphi(v) = \{w \mid \forall u \preceq w : \varphi(d^*(v, u))\},$$

where prefixes include the empty word. A faithful safety monitor M has non-error language $S_\varphi = \text{ACT}_\varphi(v_0)$ from its initial state. Let $L_M(m)$ denote its continuations from state m .

PROPOSITION 4.1 (SAFE LOOKUP CELLS FOR PURE INVARIANTS). *If a lookup at v selects a non-error state m reached together with v by a safe word in the unrestricted monitor–observer product, or a state with the same continuation language, then $\varphi(v)$ holds and $L_M(m) = \text{ACT}_\varphi(v)$.*

PROOF. Let h reach the pair (m_h, v) . Faithfulness and non-error reachability give $h \in S_\varphi$, hence $\varphi(v)$. For every suffix w ,

$$w \in L_M(m_h) \iff hw \in S_\varphi \iff w \in \text{ACT}_\varphi(v).$$

The second equivalence uses safety of the prefix h and deterministic observers: future valuations depend only on v and w . Language equivalence transfers the equality to m . $\square$

The witness need not be plant-executable or obey one-shot update restrictions. This is residual-language reasoning [13, 37], applied to the generated lookup. At activation, the adapter must choose the actual post-start key and use the same future observation rules; start is consumed once. Undefined entries and fallback initialization are outside the criterion. A true $\varphi(v)$ alone does not supply a lookup entry.

RQ2 distinguishes finite source checks from conformance at actual activations (Technical Appendix E.1).

When prior histories are part of the requirement. Some requirements intentionally refer to events before activation. In that case, the modeler declares the admitted histories, and the initialized monitor must preserve every remaining obligation of those histories. A more restrictive monitor may reject feasible updates; an empty set of admitted histories makes this check vacuous.

Our saved checks distinguish safe histories consistent with the current observation, references beginning at update entry, and whole histories that may include earlier violations. Technical Appendix B.1 defines the residual-language inclusion precisely; S3 states the conditions for each interpretation. These interpretations answer different questions and must be chosen to match the intended requirement. Cell’s pending initializer is exact for occupancy-based debt but conservative for a start-only reference that starts clear. Safe-history inclusion alone establishes neither the occupancy duty nor ACT.

4.2 Execution and Handover

The execution connection has four distinct premises (Technical Appendix F):

A1: endpoint observations preserve component/monitor states and same-labeled endpoint edges.

A2: update control disables only unselected controllables, preserves every UC event/outcome, reports the successor and eventually observes a next event.

A3: commands start quiescently and finish atomically, finitely and failure-free with an observed modeled successor.

A4: entry observes/copies the actual old state; quiescent handover installs the selected compatible state, using finite failure-free linearizable boundaries [17].

Under A1–A4, every execution maps to the linked model and reaches handover within the entry rank (Theorem F.1). History completeness places every actual activation history in its declared scope; the residual inclusion above then justifies that interpretation. A2 assumes eventual event progress, without fairness among enabled choices. Internal divergence and unobservable outcomes are excluded; rank bounds events, not time.

5 How Granularity Changes Feasibility

Merging commands removes intermediate states. We show why this can either prevent or enable completion; Section 7.2 evaluates the specified comparisons.

What is merged. Partition transfers, old stops and new starts separately into blocks. A transfer block $B \subseteq I$ simultaneously permits all targets in $\prod_{i \in B} g_i(s_i)$; a boundary block removes or initializes its monitors at the same component tuple. Other active monitors compose member-label effects, checked to commute pairwise on every monitor/residual state. Mapped precedence must remain irreflexive and acyclic. Ordinary behavior, entries, endpoints, relations, initializers and targets stay fixed; quiescence and all member guards apply. These are comparison contracts, not DUCS encodings.

LEMMA 5.1 (MANDATORY-COMMAND OBSTRUCTION). *Let q_0 be an entry and B a nonempty set of commands pending there. Suppose a safe region R contains q_0 , retains all commands in B , and is closed under every safe successor whose label is outside B . If each command in B is disabled or has an unsafe outcome at every state in R , no winning policy covers q_0 .*

PROOF. Every goal exhausts pending commands, so a winner must execute a first command in B . Safety and closure keep its prefix in R , where that step is disabled or has an environment-chosen unsafe outcome. Avoiding it yields a non-goal deadlock or infinite run, also losing. $\square$

In Cell’s one-workpiece region, joint transfer is disabled. Policy retains its initial active bits until either grouped boundary violates the interval. PC2-Rolling’s enabled joint transfer violates availability (Section 7.2.1). These results concern specified merges, not native DUCS expressiveness.

A capacity boundary. Readiness gates motivate the constructed Rolling contract [32, 39]. It starts with n ready replicas, no spares, and an interval floor of m ready replicas. A transfer boots its replicas; each becomes ready after one UC event, with no waiting loop. Service self-loops are controllable. The all-new, ready target restores capacity; service may pause during the update.

PROPOSITION 5.2 (ROLLING THRESHOLD). *For $n \geq 2$, $1 \leq m < n$ and $1 \leq k \leq n$, fixed consecutive batches of size k (with a smaller final remainder) admit safe finite completion iff $k \leq n - m$.*

PROOF. Some mandatory batch has size k . Its transfer leaves at most $n - k$ ready replicas, so $k > n - m$ violates the interval. Otherwise, transfer one batch only when all replicas are ready, disable service, and retain every UC readiness outcome before the next transfer. Readiness never falls below $n - k \geq m$. With u old and b booting replicas, $2u + b$ strictly decreases on every selected transfer and every UC event, reaching zero exactly at the target. Thus all maximal runs finish without fairness. $\square$

Why finer is not always better. Two components with one state per version share alphabet $\{x, y, tick\}$, with x, y UC and $tick$ controllable. The only UC loops are y in old 1, x in new 1, x in old 2 and y in new 2; shared labels synchronize. Thus all-old and all-new are quiescent, but either mixed product has a UC loop. With identity transfers and no requirements, either first local transfer prevents quiescence forever, so fine loses. A joint transfer skips the mixed state and wins at rank one. Every version has a *tick* loop, making both endpoints deadlock-free. Atomicity can therefore remove either a necessary or a harmful intermediate state; there is no general refinement order (Technical Appendix O.12).

6 Synthesizing and Checking an Update Policy

The procedure specializes strong-planning and on-the-fly reachability principles [8, 9, 41], retaining every queried event outcome and producing evidence for every old entry.

A bucket is $\text{Post}(q, a)$; empty means disabled. Disabling controls at a UC-enabled state removes runs without creating deadlock, preserving winning (UC priority, Lemma A.1). With quiescent goals and no fairness or controller priority, permitting commands while UC is enabled therefore

adds no winning entries. Changing fairness or priority changes the game. The proof establishes memoryless sufficiency.

6.1 Lazy Successor Generation

Algorithm 1 constructs a reachability attractor [6]. V contains discovered states, X those with all UC candidates queried, and W proved winners. A UC state needs all successors to win; a quiescent state needs one complete control bucket. Lazy queries controls incrementally, keeping unfinished cursors in Retry. Each promotion records a complete nonempty proving set and a rank above its members. Reverse incidences propagate promotions. Failure requires exhausted candidates and a promotion fixed point. Extraction chooses lower-rank controls and compatible targets; it need not minimize path length.

Algorithm 1: Lazy

Input: Typed update contract (Section 3)

Output: (σ, rk, κ) or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $rk(q) \leftarrow 0$  for  $q \in W$ ;
3 while true do
4   Add queued goals to  $W$  once with rank zero; exhaust Promote, retaining witnesses/ranks, removing winners
   from Retry and propagating predecessors;
5   if  $Q_0 \subseteq W$  then return  $(\sigma, rk|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from the goal witnesses and retained lower-rank
   choices;
6   if  $\text{Open} \neq \emptyset$  then
7     Remove  $q$  from Open, add it to  $X$ , and query all uncontrollable candidates;
8     Close the control cursor if UC-enabled; otherwise initialize it and query to the next nonempty bucket or
     exhaustion;
9     Insert  $q$  into Retry iff  $q \notin W$  and its cursor has remaining candidates;
10  else
11    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
12    Remove  $q$  from Retry; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and its
    cursor has remaining candidates;
13  Add every queried successor to  $V$ ; queue newly discovered goals for rank zero and safe non-goals for Open
    (unsafe states stay outside  $W$  and Open);
```

6.2 Correctness and Certificates

THEOREM 6.1 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), Algorithm 1 terminates. A returned (σ, rk, κ) keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

PROOF. *Coverage.* Every new safe non-goal enters Open and leaves only after expansion queries all UC candidates. At quiescence, finite control cursors retain unfinished nonwinners in Retry; promotion removes only winners. Thus expanded states have complete UC outcomes, unfinished nonwinners remain retryable, and unexpanded safe non-goals remain open.

Termination. Each state is expanded/promoted once and each candidate queried once. Reverse incidences are processed on promotion, so the finite search terminates.

Soundness. Goals start at rank zero with compatible handover states. By induction, each promotion adds a safe state above a complete nonempty proving set of previously ranked winners: every UC successor, or every outcome of one control event. Queried buckets are immutable. Extraction retains all UC events, chooses one such control bucket at quiescence, and disables other controls. Every reached non-goal is safe, has a successor, and every successor has strictly smaller natural-number rank. Consequently no maximal run is an infinite run or a non-goal deadlock, and every run reaches a matching quiescent goal. The monitor rules advance exactly the memories active over each event; induction from entry establishes every declared safety interval. Handover retains the matching new memories.

Completeness. At failure both exploration queues are empty and promotion has reached a fixed point. Some entry is in $L = V \setminus W$, and L contains no goal. Coverage ensures that each safe state in L was expanded. If it is UC-enabled, at least one UC outcome lies in L , since otherwise it would be promoted. At quiescence every control candidate is exhausted; each enabled control bucket has an outcome in L for the same reason. Against any history-dependent policy the environment can select such an outcome after every history. Disabling all events deadlocks; otherwise the resulting run reaches an unsafe state or stays outside the goals forever. Hence no winning policy covers that entry. A winning policy therefore rules out failure and, by termination, forces success. Finally, UC priority loses no winning policy: removing optional controls at a UC-enabled state removes runs without introducing a deadlock, and the restricted policy remains admissible in the original game. $\square$

Checking the returned evidence. WIN checking requires exactly $D_\sigma = \text{Reach}_\sigma(Q_0)$ as the rank domain, safety, enabled selected controls, nonempty all-outcome closure and strict decrease at non-goals. Goals require rank zero, quiescence and a compatible κ , defined exactly on $D_\sigma \cap Q_{\text{goal}}$; omitted policy entries are empty. LOSS checking requires an entry-containing, goal-free L with the UC/all-control counter-outcomes used in the completeness proof. These certify search relative to the successor and endpoint predicates [19]; Section 7 states their implementation trust boundary.

6.3 Exploring the Compiled Contract on Demand

With indexed predicates and constant-time bookkeeping, search is linear in discovered states, queried candidates and returned outcomes. This excludes preprocessing and Java sorting/scans; the reachable graph may be explored completely. Technical Appendix K gives the accounting.

7 Implementation and Evaluation

7.1 Design and Implementation

RQ1 tests granularity mechanisms; RQ2 checks games, certificates and source monitors; RQ3 compares exploration and cost on 27 adapted contracts. Auxiliary trials remain outside fixed-budget medians.

Each of the 9 inherited inputs has Base (no interval requirement) and alternative R1/R2 contracts. R1 blocks designated operational actions while flags set by any old-stop and cleared by their designated new-start are true. R2 requires selected old-stop/new-start events before their associated transfers. Actions and event pairs vary by model (S4). The separate PC2-Rolling case adds rolling calibration to a two-arm production cell. Legacy also synthesizes subsequent control and therefore solves a different objective.

Implementation. Our Modal Transition System Analyser (MTSA) extension [11] checks certificates [29]; its composition checker, Link, checks the connected old, update and new controllers,

including entries and targets. An independent oracle parses restricted Finite State Processes (FSP). Finite-JSON runs save losing membership; FSP benchmarks report only its statistics.

Use of generative AI. Generative AI assisted the research design and proof drafting, the Java solver and frontend, and the Python checkers and experiment drivers. It also assisted experiment execution, analysis and manuscript editing. We validated the results through certificate and exhaustive checks, source/automaton comparisons, saved-output checks for tables, and primary-source checks for citations. Authors retain responsibility; correlated faults from shared AI and code remain a threat. Technical Appendix P gives the tool roles.

Constructing the benchmark inputs. Nine inputs adapt eight DUCS sources; the one-arm and two-arm production-cell variants (PC1 and PC2) share a parent. PC1/PC2 preserve its repeatable, state-gated arrivals while making tool results UC. With work/output withheld, pending tool results and reset/arrival chains finish in quiescent arm states. Other adaptations shorten Railcab transfers while preserving both outcomes, omit a duplicate initial-environment transfer declaration in PowerPlant, and change safety or ordering. Technical Appendix O details these changes; equivalence is unestablished.

MTSA compiles the FSP declarations into fixed endpoints, monitors and initializer tables before exploration. Every reachable new-endpoint state is admitted. The tables combine physical states with observer values; Cell instead supplies its initializer maps directly. S3 gives the construction, and S4 retains preparation times and tuple counts for all 27 contracts.

Only Railcab transfers branch; none of the inherited inputs declares precedence. R2 instead encodes order in monitors (Technical Appendix L).

Exploration variants. All methods share entries, successors, goals, checks and terminals. On-the-fly (OTF) variants omit controls at UC-enabled states and stop when all entries win. Direct-Full constructs all reachable buckets, including those controls, then applies UC priority; auxiliary Full+UC omits them during construction. At quiescence, Lazy queries individual buckets, Eager all controls; Update-first tests O-DUCS’s update priority [18] within our solver. The query order gives priority to pending stops and transfers, and to starts after endpoint alignment. It guides exploration rather than requiring that execution order. Technical Appendix K gives the conditional priorities, queues and tie rules.

Design and measures. RQ3 uses one Windows/Xeon host, a 64 GiB heap and a 1,200 s Java Virtual Machine (JVM) cap. Each timing median requires five completions; incomplete cells retain their capped trial. Solver time includes checking, while total JVM time also includes preparation, startup and output. Exploration, policy size and peak working set provide complementary measures. Full settings, Legacy’s distinct timer, excluded Mac pilots and interruptions are retained in Technical Appendices P and O.8 and S4.

7.2 RQ1: Can Local Choices Change Feasibility?

We compare inherited inputs, constructed families and a source-derived case under Section 5’s merging rules. These comparisons do not estimate deployed prevalence.

Inherited inputs: no observed change. On nine inherited inputs, merging transfers, boundaries or both changes 0/54 Base/R1 decisions. Their total raw transfers omit Rolling’s startup phase; totality alone does not ensure safe merging. States decrease in 44 comparisons and stay equal in 10. R2 names individual events and is excluded. This null result limits observed granularity benefits.

Constructed mechanisms. The families isolate local choices, retaining both-WIN and both-LOSS controls.

Table 4 records 33/55 fine-WIN/merged-LOSS comparisons across five constructed families. Of the 33 separations, 30 are Rolling/Canary parameter points, not independent mechanisms. Rolling compares $k = 1$ with $k = n$; its 70 threshold cells are separate. Threads tests both-WIN/both-LOSS quiescence controls. Technical Appendix O.11 gives all definitions and assumption controls.

Answer. Separate commands can preserve capacity, necessary intermediate stages and compatible requirement lifetimes. Merging can instead bypass a harmful mixed state (Section 5); finer is not uniformly better. Inherited decisions do not change. PC2’s source proof establishes merged impossibility, but its merged timeout leaves the solver comparison incomplete.

Table 4. Granularity mechanisms, parameter sweeps and controls. Each pair compares fine and merged contracts of one model: Policy merges boundaries, other rows merge transfers. Counts are comparisons, not independent applications. The main total excludes Cell.

Family / control	Mechanism and parameters	Pairs	WIN / LOSS	LOSS / LOSS	WIN / WIN	Inc.
<i>Constructed families and parameter sweeps</i>						
Rolling	Readiness during startup. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Canary	Failure reports and recovery. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Policy	Audit overlap; role exclusion. 2 rule pairs	1	1	0	0	0
DB-Rolling	Role-gated replacement. $n = 3, m = 2, 3$	2	1	1	0	0
Rolling+Audit	Readiness and audit overlap. $n = 2, m = 1$; 1 audit pair	1	1	0	0	0
<i>Quiescence control</i>						
Threads (control)	Backpressure or persistent arrival. $n = 2 \dots 6$; queue 1,2	20	0	10	10	0
<i>Source-derived attempt</i>						
PC2-Rolling	Availability during calibration. 2 arms; floor 1	1	0	0	0	1
Main total		55	33	11	10	1
Cell _n (reference)	Empty-station transfer guards. $n = 2 \dots 10$	9	9	0	0	0

Outcomes read fine / merged; Inc. is unresolved. PC2’s merged solver remains TO, not LOSS. Rolling and Canary share a capacity obstruction: their sweeps supply 30 of the 33 WIN/LOSS pairs. n counts replicas/workers/stations. Canary bounds reported failures by $n - m$; otherwise m is an availability floor. Threads varies queue capacity 1,2 under both arrival regimes. Rolling+Audit’s boundary merge remains WIN and adds no pair here.

7.2.1 A Source-Derived Calibration Case. PC2-Rolling adds calibration while requiring another arm operational. Requests are controllable, completions UC, and transfers enter calibration while preserving initializer observers. Erasing calibration recovers the original operational process, without whole-problem equivalence. Endpoints retain baseline requirements plus availability; the interval additionally treats transfers as calibration entry. The 25 boundaries and two transfers have no supplied order (Technical Appendix J).

The saved policy calibrates the arms in sequence, waiting for each completion. Technical Appendix J shows one representative path; it does not prescribe a unique order for all policy branches. All 126 entries are covered, every outcome decreases rank from at most 37, and at least 1 arm stays outside calibration. Link checks the full composition. Each of the thirteen source invariants holds at its start and through the remaining start-only suffix until handover, proving ACT on reached tuples. This covers neither all initializer tuples nor all 138 occurrences; GSM below exercises ordinary actions under an active new requirement.

Whole-history obligations give a different result: only 39/126 entries admit a safe full prefix; 87 fail. Every entry also admits an earlier violating cycle that restores zero observers. Technical Appendix J preserves the complete failure partition and 38,288 initializer tuples whose safe-history nonemptiness remains unchecked.

Why the fixed merge loses. Every old entry admits both raw transfers. Before joint transfer, old plants lack calibration actions and all other steps preserve readiness. The joint transfer therefore puts both arms in calibration, clearing both interval readiness bits and reaching error. This proves impossibility, while the merged solver and fine Direct-Full both time out at 7200 s/200 GiB. Those timeouts remain unresolved solver results.

Restricted-transfer comparisons also retain both-LOSS and incomplete cases (Technical Appendix O; S1–S4).

7.3 RQ2: What Do the Checks Establish?

Answer. Table 5 maps the checks and their populations to the guarantee layers in Table 3.

Table 5. RQ2 validation populations, keyed to the scopes in Table 3.

Scope and check	Population	Established result and boundary
Game: independent games	46 fixtures, two solvers: 92 jobs	36 WIN, 48 LOSS, 8 invalid; all 84 eligible jobs pass certificate/exhaustive checks.
Invariant from start: New-requirement lookup	46 source positions, 138 occurrences; 900 non-error cells	Exact start-scoped languages under reference observations (Proposition 4.1); actual activation conformance unchecked.
From update entry: Interval initialization	135 occurrences: 46 R1, 89 R2	Exact entry-scoped initialization from source guards and stage fluents; execution is unchecked.
Game: application policies	25 Lazy WIN contracts; 125 runs	Certificates check the shared Java game. Runs save summaries; complete PC2-Rolling/Railcab exports are separate.

What is checked independently. The 92-job suite reconstructs explicit successors in Python and compares the resulting games with the solver. Every winning job passes the composition check; deliberately invalid cases omit events, deadlock or suppress uncontrollable behavior. Six separate monitor-free fixtures reconstructs the FSP endpoints, games and goals (S4.2). The larger suite shares frontend and endpoint/goal code, so its independence covers successor construction rather than the entire translation.

What the requirement checks establish. For source invariants, initial truth and transition checks establish that the monitor follows the reference observation rules. Safe lookup reachability then

supplies Proposition 4.1’s premise. Table 5 gives the distinct populations: source positions, requirement occurrences, lookup cells and saved runs.

These checks leave one connection open: whether actual activation decodes the state correctly, chooses the post-start key, follows the same observation rules and avoids fallback initialization. In particular, the adapter retains physical observers across commands while reference action predicates can reset. Conservativeness of that difference is unproved. The checks therefore establish the recorded reference semantics, rather than source semantics for every winning application policy.

For the 138 new-requirement occurrences, a separate inclusion check preserves obligations from admitted safe prior histories. Nonemptiness of the actual history scope remains unchecked except for one PC2 tuple. Update-interval requirements begin their reference at update entry. Technical Appendices E.1 and D give the full checks and interpretation conditions.

GSM after activation. For GSM’s change from $\square\text{-decode.3}$ to $\square\text{-decode.2}$, a source-derived policy covers every entry. It starts the new requirement with observer/monitor zero, permits *decode.3* and output before transfer, and rejects *decode.2*. It satisfies R1’s gap restriction and reaches a matching endpoint within twelve events, proving ACT over ordinary behavior. Technical Appendix E gives the path, rank proof and separate 45-pair entry-initialization check.

What application WIN and LOSS mean. Railcab’s Base policy covers 22 entries with 1–4 ordinary gap events; R1’s has none and retains seven interval guards. Both pass certificate/Link. Gap duties require interval requirements; WIN alone supplies none (Technical Appendix O.9).

Railcab also illustrates branching: one milestone transfer has two possible results, requiring eleven or twelve further events to reach different new-controller states. The policy completes from both, rather than selecting a favorable outcome (S4.4.3).

Exhaustive closure shows Industry/R1 loses at 42/131 entries under every order. Some workflow-validation actions violate the new monitors. The remaining document-review process permits end-less rejection, or reaches a quiescent deadlock if safe validation is withheld. Native DUCS wins its different subsequent-control problem (Technical Appendix O; S4).

7.4 RQ3: Does Lazy Reduce Exploration?

Answer. With the same UC pruning, Lazy decides 26 contracts to Eager’s 17. Direct-Full additionally changes pruning and decides 14; Lazy reduces states and queries in all 14 completed pairs. All paired decisions agree (Table 6). Update-first is faster on 8/24 paired solver medians but adds two timeouts. Lazy returns LOSS for Industry/R1 and times out on PC2/R2 at 1,200 s/64 GiB (fixed), 3,600 s/64 GiB (sensitivity) and 7,200 s/200 GiB (S4, earlier series); the limiting resource is unknown.

Legacy yields 9 WIN, 13 OOM and 5 unmeasured snapshot-cap cases. Its differing subsequent-control objective precludes same-game ratios or impossibility inferences; endpoint preservation was not compared (Technical Appendix O).

Figure 2 shows all 14 completed pairs (Technical Appendix M). Base/R1/R2 contribute 5/3/6 pairs with median Direct-Full/Lazy state ratios 111.89/11.88/2.75. These different populations neither isolate interval effects nor attribute savings to old-stop/new-start gaps. In R2, Update-first has fewer states and lower solver medians in six of eight pairs (Technical Appendix O).

Lazy’s adapter medians reach 14.221 s (PC2/R1; S4). Capped Direct-Full JVM totals are lower bounds, not solver medians or ratios. Auxiliary Full+UC timings mix builds, hosts and repeats, preventing pruning isolation; unpruned OTF is unimplemented (Technical Appendix O.10).

Scaling limits. The independent family exhibits the expected benefit of lazy construction under the default order; S4 gives every size and method.

Table 6. All 27 contracts per method. TO denotes one capped trial, not a timing median. Ratios summarize heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Faster / pairs	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—
Eager	17/0/10	0 / 17	2.04 [1.18–4,575.05]	1.60 [1.00–144.85]
Update-first	23/1/3	8 / 24	1.07 [0.19–39.76]	1.00 [0.32–6.10]
Direct-Full	14/0/13 [†]	0 / 14	9.14 [1.58–2,123.81]	2.27 [1.00–130.66]

TO: timeout; no OOM/unmeasured cases. [†] Workflow/R2 retains its interruption and later same-setting timeout (S4). Comparator/Lazy ratios are median [min–max] across completed pairs of five-run medians. Faster / pairs counts lower comparator solver medians among all completed pairs. JVM includes frontend, startup, endpoints and output; solver includes checking. Unresolved costs have no ratio.

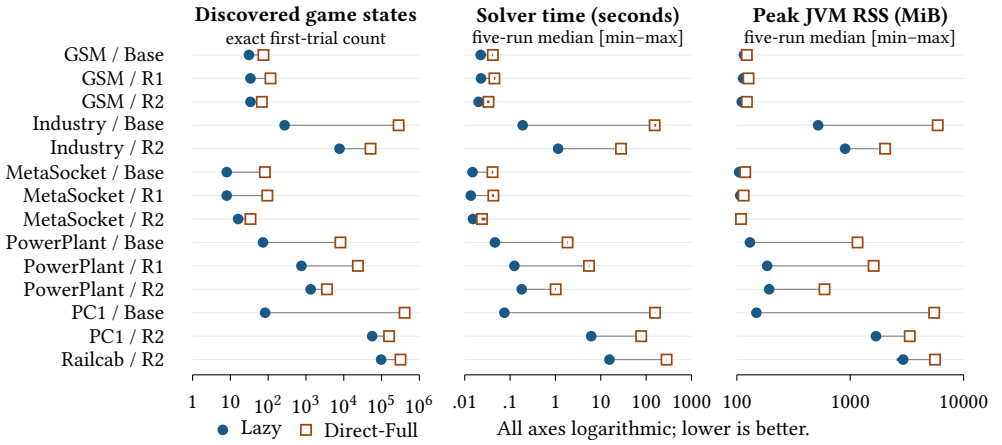


Fig. 2. All 14 completed WIN/WIN pairs. Solver time includes checking but excludes frontend, endpoint preparation and JVM startup. RSS is sampled resident memory over the whole JVM run, not solver-only or heap usage. The other 13 Direct-Full contracts time out (Figure A4) and are omitted. This compares whole procedures.

Travel reaches timeouts and out-of-memory failures under all methods, including expanded-budget timeouts. These separate trials never enter medians; Technical Appendix O.10 retains all populations and adverse outcomes.

8 Related Work

Generating update environments. Yamauchi et al. generate an update environment from pre/post environments and supplied global state-preservation transitions [44, Sec. V]. DUCS adds reconfigure edges through R without a UC-quiescence guard; a partial R can restrict its source states [34, Sec. 4.2, Defs. 3.7, 5.1]. FG gates every command and handover on global UC quiescence.

On-demand synthesis. DCS already uses on-demand component products, defers controllable alternatives, treats UC-enabled states as uncontrollable and propagates success/failure backward [9, Sec. IV]. Its 2016 formulation requires safety and eventual goal occurrence on every trace; OTF-DCS (2023) requires a marked extension of each finite controlled trace [10, Def. 4]. FG specializes

these patterns to its generated contract, with complete outcome buckets and all-entry certificates. Compositional and modular approaches structure control specifications [31, 43].

Supervisor switching. Reinitializing supervisor state can provide safe, deadlock-free switching on one plant [30, Sec. 3.3]. Workcell reconfiguration [20], bounded startup [35] and switched-system invariants [36] address related switching conditions. S5 compares their assumptions with FG-DUCS’s staged transfers and fixed endpoint.

Requirements across adaptation. Temporal adaptation [45], scenario-based updatability [14, 27] and Live Synthesis [13] address obligations across change. Our initialization uses scoped requirements [12] and residual reasoning [37]. Thuijsman and Reniers [40, Secs. 4–7] allow individual or simultaneous reconfiguration, requirements that depend on features, and state retention or reset. Their supervisor permits a marked continuation from every reachable state; FG requires every maximal update run to finish at a supplied controller.

Adaptation and operational updates. Safe adaptation graphs support minimum-cost paths [46]; architectural reconfiguration sequences preserve lifecycle and architecture constraints, including consistent recovery states [5]. Local updating, PROTEOS, workflow updating, Kitsune and network synthesis address migration and transition safety [15, 16, 26, 28, 38]. Version-consistent updating uses dynamic dependencies and selective blocking [24]. FG-DUCS addresses a different guarantee: finite handover under declared requirement lifetimes. It provides neither cost optimization, repair nor transaction-version consistency. S5 gives the detailed comparison.

9 Threats to Validity

Model and execution. Perpetual arrivals or polling can preclude quiescence. Waiting that relies on fairness is outside the model. The effects of adapting inherited inputs have not been isolated. Actual decoding, history completeness and A1–A4 remain unvalidated; repair is excluded.

Evaluation coverage. Constructed cases assume finite startup, reporting, recovery and election and may favor separations. They establish mechanisms, not deployed prevalence. Inherited inputs omit precedence and have branching transfers only in Railcab; fixtures and repeats are not independent applications. Null/LOSS results, faster Update-first cases, lower Eager/Update-first memory and Travel/PC failures bound the gains. PC2’s merged timeout remains unresolved.

Implementation and measurement. Shared authorship, AI and code risk correlated faults; independent parsing covers restricted FSP. Direct-Full is internal and Legacy solves a different objective. External solvers, randomized orders and isolated pruning remain untested. Timing uses one machine, ratios omit unresolved costs, and extended trials lack medians. RSS sampling may miss peaks and identifies neither heap occupancy nor bottlenecks. S4 retains interruptions. The exponential separation depends on the default order.

10 Conclusion

FG-DUCS compiles local transfers, requirement lifetimes and initializers into safe finite updates preserving the selected endpoint’s continuation. Merging commands can help or prevent completion. With the same UC pruning, Lazy decides 26/27 contracts to Eager’s 17; Update-first is sometimes faster. Guarantees require declared lifetimes, UC quiescence and faithful execution.

Data Availability

We created and publicly released the anonymous FG-DUCS replication package for this paper [4]. The retained evidence snapshot is available at <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-invariants-20261002>. The paper-matched release cited in [4], fgducs-c105-20261003, contains this revised main paper and paper/supplementary_material.pdf: Part I is Technical Appendix A–P; Part II is Supplement S1–S5. Its reading guide, bookmarks and continuous page numbers locate the proofs, full models and experimental records cited throughout the main text. The repository README links directly to both parts and maps paper sections to evidence. The package also provides source code, all 27 application contracts, scripts and saved results; saved-evidence checks require no new performance experiments. Dependencies come from official upstream sources.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Public anonymous replication package prepared for this paper, including Technical Appendix A–P, Supplement S1–S5, source code and saved evidence; tag fgducs-c105-20261003. <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-c105-20261003>
- [5] Fabienne Boyer, Olivier Gruber, and Damien Pous. 2013. Robust Reconfigurations of Component Assemblies. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 13–22. doi:10.1109/ICSE.2013.6606547
- [6] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*. 66–80. doi:10.1007/11539452_9
- [7] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [8] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://cdn.aaai.org/AIPS/1998/AIPS98-005.pdf>
- [9] D. Ciolek, V. Braberman, N. D'Ippolito, and S. Uchitel. 2016. *Directed controller synthesis of discrete event systems*. Technical Report 1605.09772. arXiv.
- [10] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [11] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [12] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [13] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5

- [14] Carlo Ghezzi, Joel Greenyer, and Valerio Panzica La Manna. 2012. Synthesizing Dynamically Updating Controllers from Changes in Scenario-Based Specifications. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE, Piscataway, NJ, USA, 145–154. doi:10.1109/SEAMS.2012.6224401
- [15] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147
- [16] Christopher M. Hayden, Karla Saur, Edward K. Smith, Michael Hicks, and Jeffrey S. Foster. 2014. Kitsune: Efficient, General-Purpose Dynamic Software Updating for C. *ACM Transactions on Programming Languages and Systems* 36, 4, Article 13 (Oct. 2014), 38 pages. doi:10.1145/2629460
- [17] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
- [18] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. KBSE2023-78; in Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/eng/>
- [19] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [20] Ting Jiao, Yongmei Gan, and Guochun Xiao. 2013. On the Reconfiguration of Flexible Manufacturing Workcells. In *2013 IEEE International Conference of IEEE Region 10 (TENCON 2013)*. IEEE, 1–4. doi:10.1109/TENCON.2013.6718470
- [21] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
- [22] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [23] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [24] Xiaoxing Ma, Luciano Baresi, Carlo Ghezzi, Valerio Panzica La Manna, and Jian Lu. 2011. Version-consistent Dynamic Reconfiguration of Component-based Distributed Systems. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ACM, 245–255. doi:10.1145/2025113.2025148
- [25] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://www.doc.ic.ac.uk/~jnm/book/>
- [26] Valerio Panzica La Manna. 2012. Local Dynamic Update for Component-Based Distributed Systems. In *Proceedings of the 15th ACM SIGSOFT Symposium on Component Based Software Engineering (CBSE '12)*. ACM, New York, NY, USA, 167–176. doi:10.1145/2304736.2304764
- [27] Valerio Panzica La Manna, Joel Greenyer, Carlo Ghezzi, and Christian Brenner. 2013. Formalizing Correctness Criteria of Dynamic Updates Derived from Specification Changes. In *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '13)*. IEEE, Piscataway, NJ, USA, 63–72. doi:10.1109/SEAMS.2013.6595493
- [28] Jedidiah McClurg, Hossein Hojjat, Pavol Černý, and Nate Foster. 2015. Efficient Synthesis of Network Updates. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. ACM, New York, NY, USA, 196–207. doi:10.1145/2737924.2737980
- [29] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [30] Rômulo Meira-Góes and Stéphane Lafortune. 2020. Moving Target Defense based on Switched Supervisory Control: A New Technique for Mitigating Sensor Deception Attacks. *IFAC-PapersOnLine* 53, 4 (2020), 317–323. doi:10.1016/j.ifacol.2021.04.031
- [31] Sahar Mohajerani, Robi Malik, and Martin Fabian. 2014. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Trans. Automat. Control* 59, 1 (2014), 150–162. doi:10.1109/TAC.2013.2283109
- [32] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
- [33] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '16)*. ACM, New York, NY, USA, 96–107. doi:10.1145/2897053.2897056

- [34] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
- [35] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [36] Spyros Reveliotis and Zhennan Fei. 2017. Invariant-Based Supervisory Control of Switched Discrete Event Systems. *IEEE Trans. Automat. Control* 62, 2 (2017), 921–927. doi:10.1109/TAC.2016.2560536
- [37] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
- [38] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE ’22)*. ACM, New York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [39] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [40] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [41] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [42] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D’Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733
- [43] W. Murray Wonham and Peter J. Ramadge. 1988. Modular Supervisory Control of Discrete-Event Systems. *Mathematics of Control, Signals, and Systems* 1, 1 (1988), 13–30. doi:10.1007/BF02551233
- [44] Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2020. Method for Low-Cost Environment Partitioning Modeling in Dynamic Update. In *Proceedings of the 2020 IEEE Third International Conference on Artificial Intelligence and Knowledge Engineering (AIKE ’20)*. IEEE, Piscataway, NJ, USA, 183–187. doi:10.1109/AIKE48582.2020.00036
- [45] Ji Zhang and Betty H. C. Cheng. 2006. Using Temporal Logic to Specify Adaptive Program Semantics. *Journal of Systems and Software* 79, 10 (2006), 1361–1369. doi:10.1016/j.jss.2006.02.062
- [46] Ji Zhang, Betty H. C. Cheng, Zhenxiao Yang, and Philip K. McKinley. 2005. Enabling Safe Dynamic Component-Based Software Adaptation. In *Architecting Dependable Systems III*, Rogério de Lemos, Cristina Gacek, and Alexander Romanovsky (Eds.). Lecture Notes in Computer Science, Vol. 3549. Springer, Berlin, Heidelberg, 194–211. doi:10.1007/11556169_9